

Fonctions au nombre de paramètres variable

Exemple d'utilisation

Ce tutoriel présente un exemple d'utilisation d'une fonction avec un nombre de paramètres variable, que l'on appelle une fonction *variadic*.

1 Présentation des macros

Pour définir une fonction *variadic*, il faut finir la déclaration des paramètres par «...». Il doit y avoir un paramètre avant et aucun après. Par exemple :

```
int add(int n, ...)
```

L'accès aux paramètres passe par l'utilisation d'une `va_list` définie dans `stdarg.h` puis de macros permettant de parcourir la liste.

Dans un premier temps, on initialise la variable à l'aide de `va_start` en spécifiant le paramètre précédent (dans l'exemple ci-dessous, le paramètre «a»). Par exemple :

```
va_list lstPar;  
va_start(lstPar, a);
```

Pour récupérer le paramètre suivant, on utilise la macro `va_arg` qui prend en paramètre la liste et le type du paramètre. Une fois tous les paramètres parcourus, on utilise la fonction `va_end` pour clôturer la liste.

2 Exemples d'utilisation

Voici un exemple de fonction qui prend un nombre d'entiers puis une liste d'entiers. Elle calcule la somme des entiers.

```
int add(int nb, ...) {  
    va_list lstPar;  
    int sum = 0, i;  
  
    va_start(lstPar, nb);  
    for(i = 0; i < nb; i++) {  
        sum += va_arg(lstPar, int);  
    }  
    va_end(lstPar);  
  
    return sum;  
}
```



Le type doit être connu à l'avance dans la fonction et comme le compilateur ne peut pas vérifier, le comportement d'une fonction pour laquelle on ne passe pas les paramètres corrects a un comportement indéfini.

Pour appeler la fonction, il suffit de faire :

```
printf("Add:_%d\n", add(5, 1, 2, 3, 4, 5));
```

Il est également possible de ne pas spécifier le nombre d'arguments si on utilise des pointeurs. Il suffit de terminer la liste par NULL par exemple. La fonction suivante concatène un nombre quelconque de chaînes de caractères.

```
char* concat(char *str, ...) {
    char *result, *ptr;
    va_list lstPar;
    size_t length = 0;

    // Détermine la longueur de la chaîne résultante
    va_start(lstPar, str);
    ptr = va_arg(lstPar, char*);
    while(ptr != NULL) {
        length += strlen(ptr);
        ptr = va_arg(lstPar, char*);
    }
    va_end(lstPar);

    if((result = malloc(sizeof(char) * (length + 1))) == NULL) {
        fprintf(stderr, "Error_allocating_string\n");
        exit(EXIT_FAILURE);
    }

    // Concatène les chaînes
    va_start(lstPar, str);
    ptr = va_arg(lstPar, char*);
    length = 0;
    while(ptr != NULL) {
        strcat(&result[length], ptr);
        length += strlen(ptr);
        ptr = va_arg(lstPar, char*);
    }
    va_end(lstPar);

    return result;
}
```

À noter que cette fonction parcourt deux fois la liste : la première pour déterminer la longueur totale de la chaîne et une seconde pour la concaténation à proprement parler. À chaque fois, on utilise `va_start` et `va_end`.

Pour appeler la fonction, il suffit de faire :

```
char *str = concat("Coucou_", "tout_", "le_", "monde.", NULL);  
printf("Final_string:_%s\n", str);  
free(str);
```